

Sistemas Distribuídos — Aula 19

Algoritmos Distribuídos II: Snapshot Distribuído e Detecção de Terminação · 20/10 · Prof. Leandro Borges

O problema do snapshot distribuído

Na aula anterior vimos como um conjunto de processos pode concordar sobre um único valor mesmo diante de falhas — o problema do consenso. Agora tratamos de um problema diferente, mas igualmente fundamental: como capturar uma fotografia consistente do estado global de um sistema distribuído, sem interromper sua execução. O estado global é a combinação do estado interno de cada processo com o conteúdo de cada canal de comunicação — as mensagens que já foram enviadas, mas ainda não chegaram ao destino. Conhecer esse estado é essencial para depuração de sistemas distribuídos, para implementar checkpointing (salvar o progresso de um cálculo longo para poder retomá-lo depois de uma falha), para detectar situações de impasse (deadlock) e para algoritmos de coleta de lixo distribuída.

O obstáculo central é que não existe, em um sistema distribuído real, um relógio global compartilhado nem qualquer forma de pausar todos os processos exatamente no mesmo instante de tempo físico. Se cada processo simplesmente registrasse seu próprio estado local no momento em que recebesse um pedido de captura, o resultado poderia facilmente ser inconsistente: uma mensagem poderia aparecer duplicada em dois lugares ao mesmo tempo, ou desaparecer completamente da fotografia, mesmo estando de fato em trânsito pela rede no instante da captura.

Por que um snapshot ingênuo é inconsistente

Para entender o problema concretamente, imagine dois processos, A e B, ligados por um canal de comunicação, e suponha que cada um registre seu estado local de forma independente, sem nenhuma coordenação. Um primeiro tipo de inconsistência surge quando B registra seu estado antes de A registrar o seu, mas A já enviou uma mensagem para B que ainda não chegou. Se essa mensagem chegar depois da captura de B, ela vai aparecer no estado de B como algo recebido, mas em nenhum lugar do snapshot ela aparecerá como enviada — uma mensagem “fantasma”, recebida sem nunca ter sido enviada, o que é logicamente impossível em qualquer execução real do sistema.

O segundo tipo de inconsistência é o oposto: A envia uma mensagem e só depois registra seu próprio estado, mas B já havia registrado o seu antes de receber essa mensagem. Nesse caso, o snapshot mostra a mensagem como enviada por A, mas ela não aparece em lugar nenhum como recebida por B — ela some da fotografia, mesmo estando de fato em trânsito pela rede no momento em que a captura estava sendo feita. Um snapshot correto precisa evitar os dois problemas ao mesmo tempo: nada pode ser contado duas vezes, e nada pode desaparecer.

Chandy-Lamport: marcadores e captura dos canais

O algoritmo de Chandy-Lamport, publicado em 1985 por K. Mani Chandy e Leslie Lamport (o mesmo autor do relógio lógico e, mais tarde, de Paxos), resolve esse problema com uma ideia elegante: usar uma mensagem especial, chamada marcador (marker), que se propaga pela mesma rede que carrega as mensagens normais da aplicação, sem exigir nenhum relógio global. Qualquer processo pode iniciar a captura: ele registra seu próprio estado local naquele instante e, imediatamente depois, envia um marcador por todos os seus canais de saída, antes de enviar qualquer outra mensagem de aplicação gerada depois do início da captura.

A regra que qualquer processo segue ao receber um marcador é o que faz o algoritmo funcionar: se esse for o primeiro marcador que o processo recebe (em qualquer canal), ele registra

imediatamente seu próprio estado local — mesmo que ainda não tivesse participado da captura — e começa a gravar, como o estado daquele canal específico, todas as mensagens de aplicação que chegarem por ele até que um marcador também chegue por esse canal. Quando um marcador chega em um canal que já estava sendo monitorado, ele simplesmente fecha o registro daquele canal: tudo que chegou entre o registro do estado do processo e a chegada do marcador é, por definição, o conteúdo do canal na fotografia. Quando todo processo já registrou seu estado e todo canal já recebeu seu marcador, o snapshot está completo — distribuído entre os processos, sem nunca ter existido em um único lugar centralizado.

O que um snapshot consistente garante

O resultado produzido pelo algoritmo de Chandy-Lamport é chamado de corte consistente: uma coleção de estados locais e estados de canais tal que, se um determinado evento está incluído na fotografia, todo evento que o causou (na ordem causal definida pela troca de mensagens) também está incluído. Isso garante que nenhum efeito apareça sem sua causa correspondente — a propriedade mínima para que o snapshot faça sentido logicamente, mesmo sem representar nenhum instante único do tempo físico real.

É importante entender exatamente o que essa fotografia representa: ela não corresponde necessariamente a nenhum instante que de fato ocorreu durante a execução real do sistema, mas sim a um estado global que poderia ter ocorrido, dada a mesma sequência causal de eventos. Essa garantia é suficiente na prática porque a maioria das propriedades que queremos verificar em um sistema distribuído — como “o sistema está em deadlock” ou “a soma total de um valor conservado permanece constante” — é estável: se é verdadeira em algum corte consistente, também seria verdadeira em qualquer execução real equivalente. E, fundamentalmente, tudo isso acontece sem jamais parar a computação: os processos continuam trabalhando normalmente durante toda a captura, que roda de forma inteiramente concorrente e descentralizada.

Detecção de terminação distribuída

O segundo problema desta aula é, de certa forma, um caso particular de conhecer o estado global: saber quando um cálculo distribuído terminou. Um cálculo distribuído termina quando todos os processos envolvidos estão passivos — sem nenhum trabalho pendente a executar — e não existe nenhuma mensagem em trânsito entre eles. A dificuldade é que nenhum processo, isoladamente, consegue saber disso: um processo pode estar passivo neste exato momento e ser reativado, um instante depois, por uma mensagem enviada por outro processo que ainda está a caminho pela rede. Sem visibilidade sobre os canais de comunicação, qualquer processo isolado poderia declarar término prematuramente e estar simplesmente errado.

Assim como no problema do snapshot, a dificuldade de fundo é a ausência de um relógio global e de um coordenador central que enxergue todos os processos e canais simultaneamente. Afirmar coletivamente que “o cálculo terminou” exige, portanto, um protocolo adicional, executado pelos próprios processos participantes, que combine informação local de cada um (está ativo ou passivo? quantas mensagens enviei? quantas recebi?) até que seja possível concluir, com segurança, que nenhuma atividade futura é mais possível em lugar nenhum do sistema.

Duas soluções clássicas: Dijkstra-Scholten e wave algorithms

Uma das primeiras soluções para esse problema, proposta por Edsger Dijkstra e Carel Scholten em 1980, constrói dinamicamente uma árvore de dependências enraizada no processo que iniciou o cálculo (a chamada computação difusora). Toda vez que um processo envia uma mensagem de trabalho para outro, ele se torna o “pai” temporário desse processo na árvore; toda vez que um processo recebe um reconhecimento (ack) de todos os trabalhos que delegou e ele próprio já está passivo, ele envia um ack de volta ao seu próprio pai e “sai” da árvore. O cálculo termina, com certeza, no exato momento em que a árvore inteira colapsa de volta até a raiz — o processo iniciador recebe o último ack e sabe, sem qualquer ambiguidade, que nenhuma atividade resta em

lugar nenhum do sistema. A elegância do método está em usar apenas informação estritamente local (quem enviei trabalho, de quem recebi ack) para inferir uma propriedade inteiramente global.

Uma família alternativa de soluções, os chamados wave algorithms (algoritmos de onda), inclui abordagens como o anel de tokens: em vez de rastrear uma árvore de dependências, um token especial circula continuamente por um anel lógico que passa por todos os processos, coletando a informação de estado (ativo ou passivo) de cada um. Só quando o token consegue completar uma volta inteira encontrando todo processo passivo e nenhuma mensagem de aplicação pendente em nenhum canal, o término pode ser declarado com segurança — se qualquer processo se tornar ativo durante o percurso, a volta é invalidada e uma nova rodada do token é necessária. A ideia central, comum a todos os wave algorithms, é fazer uma “onda” de verificação percorrer repetidamente todo o sistema até confirmar uma condição de quietude global, sem depender de nenhuma estrutura hierárquica fixa como a árvore de Dijkstra-Scholten.

Essas técnicas não são apenas exercícios teóricos: snapshots consistentes como o de Chandy-Lamport são a base de mecanismos de checkpointing em simulações científicas e sistemas de processamento em cluster de longa duração, permitindo retomar um cálculo interrompido por falha sem reprocessar tudo desde o início. Detecção de terminação, por sua vez, é o que permite a frameworks de processamento distribuído em massa — e a coletores de lixo distribuídos — saber com segurança quando uma fase de trabalho realmente acabou e é seguro avançar para a próxima, mesmo com milhares de processos operando de forma independente. Na próxima aula, veremos como sistemas distribuídos usam ideias semelhantes de coordenação para resolver um terceiro problema central: manter múltiplas réplicas dos mesmos dados consistentes entre si.

Referências

CHANDY, K. M.; LAMPORT, L. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Transactions on Computer Systems*, v. 3, n. 1, 1985.

DIJKSTRA, E. W.; SCHOLTEN, C. S. Termination Detection for Diffusing Computations. *Information Processing Letters*, v. 11, n. 1, 1980.

TANENBAUM, A. S.; VAN STEEN, M. *Sistemas Distribuídos: Princípios e Paradigmas*. São Paulo: Pearson.